

M3. Control inteligente de un cruce semaforizado (cruce asignado)

Empezar tarea

- Fecha de entrega Martes a las 23:59
- Puntos 100
- Entregando un cuadro de entrada de texto, una URL de página web, o una carga de archivo
- Tipos de archivo docx, doc, y pdf
- Disponible 29 de ago en 1:00 - 2 de sep en 23:59

Propósito

Aplicar una técnica de decisión (heurística, Markov Decision Process, Q-learning o búsqueda) para **ajustar el plan semafórico y mejorar el desempeño** del cruce (menor tiempo de espera, menor cola, mayor throughput).

Instrucciones

Entorno base

Usar el **notebook del cruce** (autos en cuatro aproximaciones + semáforos con R/Y/G). Mantener generación Poisson por aproximación y métricas:

<https://colab.research.google.com/drive/1bYH5n2WxyGMEB45SmFjQ-37tR5LISzrh?usp=sharing>

→ <https://colab.research.google.com/drive/1bYH5n2WxyGMEB45SmFjQ-37tR5LISzrh?usp=sharing>

- Delay promedio $\bar{D} = \frac{1}{n} \sum_i (t_i^{salida} - t_i^{llegada})$
- Cola máxima por aproximación $Q_d^{\max} = \max_t Q_d(t)$
- Throughput = # vehículos descargados

¿Qué debes hacer? (individualmente para la ruta asignada por grupo)

1. Heurística por utilidad/recompensa (reglas adaptativas)

- Definir una **función objetivo** (p.ej. $J = \alpha \bar{D} + \beta \sum_d Q_d^{\max} - \gamma \text{throughput}$).
- Política: cada Δ segundos, **extender/cambiar** fase si reduce

Error al analizar MathML: error on line 1 at column 80: Opening and ending tag mismatch: br line 1 and n

J (evaluado con una **heurística** simple, ej. “si $\sum Q_{NS} > \sum Q_{EW} + \theta$ prolonga NS”).

- Mostrar cómo J mejora vs. plan fijo.

2. MDP (Value Iteration/Policy Iteration simplificado)

- Estados s : $(Q_N, Q_S, Q_E, Q_W, \text{fase})$ discretizados (ej. colas por tramos 0–2–4–6+).
- Acciones a : {extender fase actual, **cambiar** a la otra}.
- Recompensa $R(s, a, s') = -(\bar{Q} + \lambda \text{wait})$.
- Usar **Ecuación de Bellman**:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

- Aproximar $P(s'|s, a)$ con simulación (“modelo empírico”): desde s y acción a , avanzar algunos ticks y estimar promedios.

3. Q-learning (modelo-libre)

- Q-estado: igual que MDP (discretizado).
- Actualización:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

- Exploración ϵ -greedy. Reportar la **curva de aprendizaje** (retorno acumulado por episodio) y desempeño final vs. plan fijo.

4. Búsqueda (árbol de decisiones cortoplacista)

- En cada decisión, expandir un **árbol de profundidad H** con acciones {extender, cambiar}.
- Simular transiciones (rollouts) y **evaluar hojas** con la función objetivo J .
- Elegir la raíz con **Best-First / A*** (si defines heurística) o **MCTS** (si te animas).
- Mostrar mejora frente a plan fijo.

Tip: Para **discretizar colas**: 0, 1–2, 3–5, 6+. Para tiempo: decisiones cada 5–10 s. Mantén el **ámbar** y **all-red fijos**.

Entregables

1. Notebook (ipynb) con:

- Definición clara de la técnica (heurística/MDP/Q/Q-search).
- Código, gráficos, tablas con métricas.

- Comentarios breves (en el documento a entregar) explicando decisiones.

2. Doc o PDF de maximo 5 páginas:

- Problema, función objetivo, método elegido y por qué.
- Resultados (tablas/gráficas) y discusión.
- Conclusiones: cuándo tu método mejora o no.

Rúbrica (100 pts)

- **Correctitud técnica (30):** implementación consistente con la técnica elegida.
- **Diseño experimental (20):** escenarios, semillas, métricas claras.
- **Mejora cuantitativa (20):** evidencia de mejora vs. plan fijo (aunque sea parcial, bien discutida).
- **Claridad y explicación (20):** PDF y comentarios en código.
- **Calidad de código (10):** legible, modular, sin “magia” oculta.

Bloques guía (pseudocódigo)

Heurística adaptativa

```
if sumQ('NS') > sumQ('EW') + theta and phase=='NS' and t_in_phase < Gmax_NS:
    action = 'extend'
elif phase=='NS' and t_in_phase >= Gmin_NS:
    action = 'switch'
# aplicar action y simular Δs
```

Value Iteration (modelo empírico)

```
for it in range(K):
    for s in S:
        V[s] = max_a mean_over_rollouts( R(s,a,s') + gamma*V[s'] )
```

Q-learning

```
a = epsilon_greedy(Q[s])
r, s_next = simulate(a, Δ)
Q[s,a] += alpha*( r + gamma*max(Q[s_next,*]) - Q[s,a] )
```

s = s_next

Búsqueda con rollouts

```
def evaluate(node, depth):
    if depth==H: return -J(state_of(node))
    best = -inf
    for a in actions:
        s2 = simulate(state, a, Δ)
        best = max(best, evaluate(s2, depth+1))
    return best
```

Para que adaptes el **cruce** y veas la **animación HTML** (`HTML(anim.to_jshtml())`) debes indicarles **exactamente qué celdas/partes tocar** del notebook de la intersección con semáforos que ya les diste.

Abajo te dejo las “palancas” con línea-guía y ejemplos listos para pegar.

1) Parámetros generales (Celda 2: `params`)

Qué cambiar:

- **Tiempos de semáforo** (verde/ámbar/all-red).
- **Tasas de arribo** por aproximación.
- **Geometría base** (largo `L`, ancho de carril `w`).

```
params.update({
    'green_ns': 30,    # o 90, etc.
    'green_ew': 45,
    'yellow': 4,
    'all_red': 2,
    'lambda_N': 0.08, 'lambda_S': 0.12, 'lambda_E': 0.05, 'lambda_W': 0.15,
    'L': 100.0, # extiende el dibujo si tu croquis es más largo
    'w': 3.5,
})
```

2) Fases del semáforo (Celda 3: `FourWaySignals`)

Qué cambiar:

- Si tu cruce **no es simétrico** o quieres **3+ fases** (p.ej., fases exclusivas de giro), edita la lógica de `lights()` y del temporizador en `step()`.

Ejemplo: fase 0 = NS (verde), fase 1 = giros E/W (amarillo/verde breve), fase 2 = EW (verde):

```

class FourWaySignals(ap.Agent):
    def setup(self, green_ns, green_ew, yellow, all_red):
        self.g_ns, self.g_ew, self.y, self.ar = int(green_ns), int(green_ew), int(yellow), int(all_red)
        self.phase, self.sub, self.t_in = 0, 'G', 0

    def lights(self):
        L = {d:'R' for d in ['N','S','E','W']}
        if self.phase == 0:          # NS rectos
            L['N'] = L['S'] = self.sub
        elif self.phase == 1:        # fase intermedia (ej. proteger giros de E/W)
            L['E'] = L['W'] = 'Y' if self.sub=='G' else self.sub
        else:                        # EW rectos
            L['E'] = L['W'] = self.sub
        return L

    def step(self):
        # avanza subestados y cambia self.phase cuando toque (ajusta duraciones por fase)
        # (idéntico a tu versión, pero con ifs por fase 0/1/2 y sus tiempos)
        ...

```

Si el cruce es **T** (tres brazos): deja en rojo permanente la aproximación que no existe (p.ej. `L['W']='R'` siempre).

3) Geometría y carriles / líneas de alto (Celda 4: `Car.setup`)

Qué cambiar:

- **Origen** (spawn), **dirección**, **línea de alto** y **meta** por aproximación.
- Esto define **por dónde “corren” los autos**.

Ejemplo (modifica coordenadas según tu croquis):

```

if origin == 'N':
    self.pos = np.array([-w/2, L]) # spawn
    self.dir = np.array([0, -1])   # hacia el sur
    self.stopline = np.array([-w/2, +5]) # línea de alto más lejos del centro
    self.goal = np.array([-w/2, -L]) # salida
# Repite para 'S','E','W' con sus posiciones reales

```

Si el cruce tiene **ángulo distinto a 90°** o hay **desplazamientos**, ajusta las `self.pos`, `self.dir`,

`self.stopline` y `self.goal` acorde al croquis (usa papel milimétrico/escala simple).

4) Lógica de arribos por aproximación (Celda 5: `spawn_poisson`)

Qué cambiar:

- **Qué flujos** entran (N/S/E/W) y **con qué intensidad**.
- Si un cruce **no tiene** cierta entrada, **no la spawnnees**.

```
def step(self):
    # Arribos activos
    self.spawn_poisson('N', self.p.lambda_N) # comenta si no hay norte
    self.spawn_poisson('S', self.p.lambda_S)
    # por ejemplo, solo E y S en un cruce en T:
    # self.spawn_poisson('E', self.p.lambda_E)
    # self.spawn_poisson('W', self.p.lambda_W) # desactiva si no existe
    ...
```

5) Dibujo/Animación del cruce (Celda 6: `draw_intersection` y `my_plot`)

Qué cambiar:

- **Pintar carriles y líneas de alto** exactamente como tu croquis.
- **Ubicación de focos** (círculos de colores) para cada aproximación.

```
def draw_intersection(ax, L, w):
    ax.clear(); ax.set_xlim(-L,L); ax.set_ylim(-L,L); ax.set_aspect('equal')
    ax.set_xticks([]); ax.set_yticks([])
    # Rectas de calzada: ajusta posiciones para tu croquis
    ax.add_patch(plt.Rectangle((-L, -w), 2*L, 2*w, color='#e0e0e0')) # horizontal
    ax.add_patch(plt.Rectangle((-w, -L), 2*w, 2*L, color='#e0e0e0')) # vertical
    # Líneas de alto: mueve/rota según el plano
    ax.plot([-6,6], [+5, +5], color='yellow', lw=2) # norte
    ax.plot([-6,6], [-5, -5], color='yellow', lw=2) # sur
    ax.plot([+5, +5], [-6,6], color='yellow', lw=2) # este
    ax.plot([-5, -5], [-6,6], color='yellow', lw=2) # oeste
```

Y en `my_plot` actualiza **las coordenadas** de luces:

```
locs = {'N':(-w/2, +w), 'S':(+w/2,-w), 'E':(+w, +w/2), 'W':(-w, -w/2)}
# mueve estas tuplas (x,y) a donde estén tus semáforos en el dibujo
```

6) Ver la animación HTML (Celda 7)

No cambia: solo recuerda **re-ejecutar** la celda del **modelo** si tocaste alguna clase.

```
fig, ax = plt.subplots(figsize=(6,6))  
model = FourWayModel(params) # <- vuelve a crear el modelo tras tus cambios  
anim = ap.animate(model, fig, ax, my_plot)  
from IPython.display import HTML  
HTML(anim.to_jshtml())
```

Guía rápida para tus 6 cruces

1. **Abrir su notebook del equipo (copia única por cruce).**
2. **Editar en orden:**
 - Celda 2 → tiempos y flujos.
 - Celda 3 → fases si necesitan 3+ fases (o T/rotonda).
 - Celda 4 → posiciones (spawn/stop/goal) por aproximación.
 - Celda 5 → activar solo las entradas que existen y sus λ .
 - Celda 6 → dibujo: carriles, líneas de alto, focos.
3. **Re-crear el modelo** (celda 7) y **ver la animación HTML**.
4. (Opcional) **Exportar** CSV (`signal_timeline` y posiciones) si lo llevarán a Unity.